CS482 Software Project:
5-Draw Poker

Cyrus McCollim, Heloisa Albuquerque, Charles Humphreys, Sean Triplett

2026-11-09

1 Client Information

- Client name: Dr. Megan Olsen
- Client title: Professor
- Client email address: mmolsen@loyola.edu
- Client employer: Loyola University Maryland
- How you know the client: Professor

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
1	Aesthetics	Design	The color scheme is red and assets used for backgrounds, buttons, cards, and other elements are for a mature audience.
3	Time Outs	Gameplay	System automatically kicks users taking too long on their turn and deletes inactive rooms.
4	Money	Reliability	The system accurately processes and updates balances and payouts without loss (including during multiplier events).
6	Rate Limits	Security	System has rate limits for sensitive actions like guest sign in, lobby creation, clicking ads, sending messages.
7	Availability	Reliability	The site is hosted to be highly available for play at most times.
8	Mobile	Gameplay	The site and game are functional on mobile devices.
10	Sound	Design	System plays music and sound effects for game interactions and events.

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
S1	Game Initialization	2	As an Player, I want us to be dealt 5 cards after putting our antes into the pot, so that the first round can begin.
S2	Game Betting	2	As a/an Player, I want to be able to Check, Call, Bet/Raise, or Fold, so that I can make a decision during a betting round.
S3	Game Card Draw	2	As a/an Player, I want to select between 0 and 5 cards to discard, so that I can replace them with new cards from the deck during the draw round.
S4	Game Ending	2	As a/an Player, I want all player hands to be compared after the final betting round, so that the pot is awarded to the winner.
S5	Bot Play	5	As a/an Player, I want to add bots to the room, so that I can play 5 Card Draw Poker when not enough people are online.

S6	Sign Up	2	As a/an Player, I want to create an account, so that I can save my progress.
S7	Sign In / Sign Out	1	As a/an Player, I want to sign in / sign out to my account, so that I can pick up where I left off.
S8	User Settings	3	As a/an Player, I want to update my own profile, so that I can customize how I appear to others.
S9	Guest Player	1	As a/an Player, I want to login as a guest, so that I can play without an account.
S10	View Balance	1	As a/an Player, I want to view my balance, so that I can know how much I have won.
S11	View My History	2	As a/an Player, I want to see my game history, so that I can see my performance over time.
S12	Room Creation	2	As a/an Player, I want to create a room, so that I can play with others.
S13	Room Settings	3	As a/an Admin, I want to adjust room settings, so that I can change the max players, set minimum bets, and configure raise per X rounds.
S14	Room Joining	2	As a/an Player, I want to join a room, so that I can play with others.
S15	Chat Feature	5	As a/an Player, I want to be able to message in a chat room, so that I can communicate with other players during a game.
S16	Contact Admin	1	As a/an Player, I want to message the admin, so that I can report any problems.
S17	Advertisements	1	As a/an Player, I want to click on ads, so that I can get more in game money.
S18	Leaderboard	1	As a/an Player, I want to see a leaderboard of the top balance players, so that I can see where I stand.
S19	Daily Rewards	1	As a/an Player, I want to get daily rewards, so that I have a reason to log in every day.
S20	Rank Badge	1	As a/an Player, I want to receive a badge when I reach the top of the leaderboard, so that my profile is distinguished as a top player.
S21	Cosmetic Shop	5	As a/an Player, I want to buy cosmetic items, so that I have something to spend my in game money on.
S22	Promotional Event	1	As a/an Admin, I want to trigger special events with chip multipliers, so that I can boost player engagement.
S23	Remove Users	2	As a/an Admin, I want to delete a user, so that I can remove malicious players.
S24	Reset Password	1	As a/an Admin, I want to reset a user's password, so that a player is not locked out if they forget it.
S25	View All Profiles	2	As a/an Admin, I want to see all player game histories, so that I can inspect games for problems.
S26	Modify Chips	1	As a/an Admin, I want to modify a player's balance, so that I can reimburse them for problems.
S27	Adjust Volume	1	As a/an Player, I want to change volume levels for music and sound effects, so that I can choose what I hear.
S28	Friends List	3	As a/an Player, I want to have a friends list that I can manage, so that I can play with my friends again, add them, or remove them.
S29	Advertisements	1	As a/an Admin, I want to create ads, so that players can have ads to click on to get in game money.
	Total:	57	

Table 2: Functional Requirements (User Stories)

3.3 AI Analysis on Functional Requirements

[After you receive your revised User Stories in CS482, or after your first delivery of Stories on CS496, you should use AI as a senior software engineer to help you find any missing or hidden requirements.] [You must provide which LLM you use, which prompt you used, and most importantly *reflect* on the results the LLM gave back. Just because the LLM suggested something, it does not mean it was a valid suggestion and that you should follow blindly. Use your knowledge and reason on which ones are reasonable to add to your requirements and which ones are not.]

4 System Design

4.1 Architecture

We will use the MVC architecture to structure this application. Two servers will be hosted for the frontend and backend respectively. The frontend will serve the HTML/CSS/JS which includes graphics, menus, buttons, animations, and interface logic. The backend will be the source of truth by validating requests, updating game states, and interacting with the database. This should help with persistent data and real-time multiplayer interactions. The backend separates the game engine from the network and database layers to ensure that the card mechanics for 5-card poker can be unit tested. The main modular components for the application include Auth, Player, Profile, Friends, Admin, Lobby, Room, Chat, Game State, Hand Evaluator, Bot, Wallet, Shop, History, Leaderboard, Ads, and Events.

4.2 Diagrams

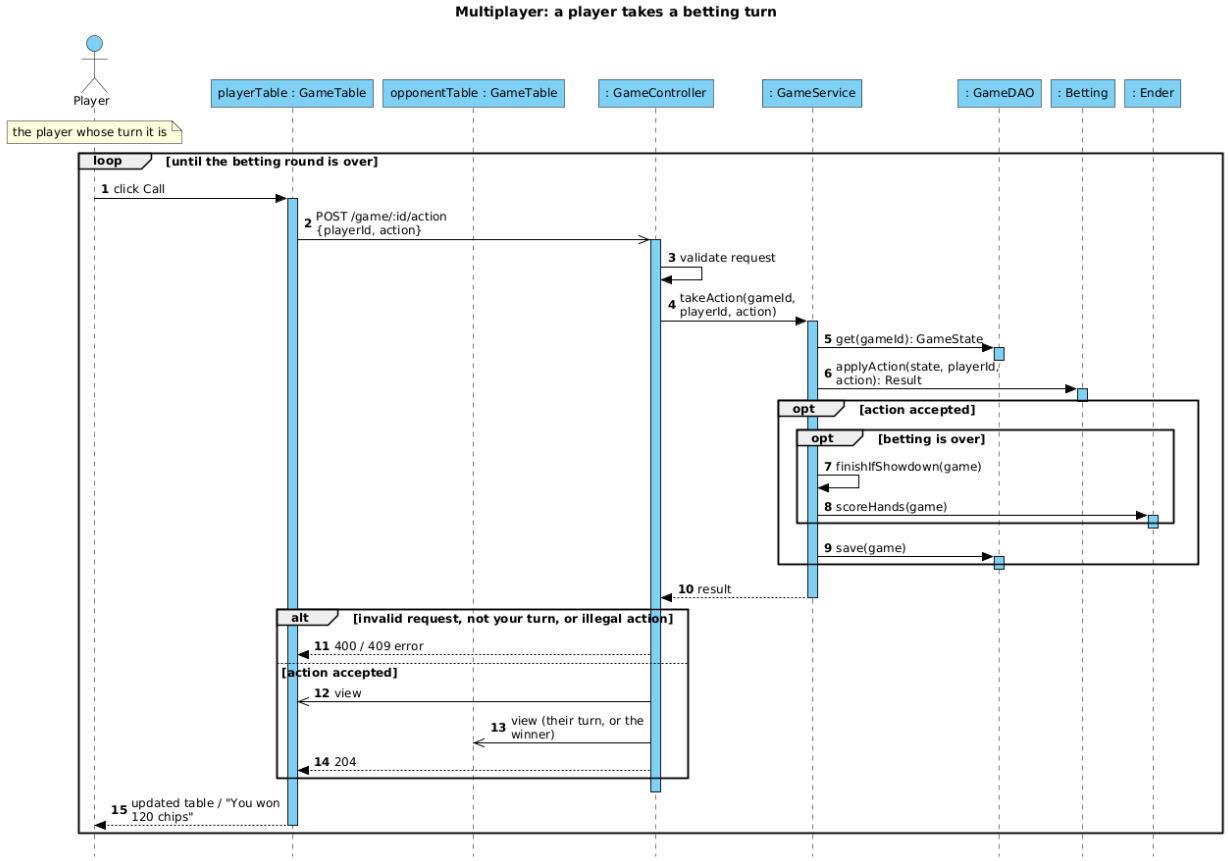


Figure 1: Sequence diagram of a multiplayer betting turn in 5 Card Draw.

Figure 1 shows one multiplayer scenario. The player whose turn it is takes a betting action (call). The browser then sends the action to the server as an asynchronous HTTP request. The **GameController** validates the request and passes it to the **GameService**, which loads the game from the in-memory **GameDAO** and asks the **Betting** to apply it. The server is authoritative, so it rejects an action that is taken out of turn or not allowed, and the player sees the error. When an action is accepted, the new state is saved, and it ends the betting /textbfEnder scores the hand and pays out the pot. Every player recieves their own updated view of the table over socket.io (asynchronous messages), so no player is ever sent another player's cards. The loop repeats with the next player until the betting round is over. The database is not shown because a betting turn does not use it and the running game lives in memory on the server.

4.3 Technology

For our programming language, we chose **TypeScript** for both the front and back end because of its strict type enforcement makes unit testing and error checking easier. For our front end, we will use **React** bootstrapped with **Next.js** (written in TypeScript) for our UI, which has built-in file-based routing. For our backend, we will use **Node.js** with **Express** and a **REST API** for accounts, friends, tables, and adverts compiled from typescript. For real-time communication, we will use **Socket.IO** to manage live gameplay updates. Each poker table is a room, and Socket.IO's built-in reconnection handles dropped players. It also comes shipped with its own TypeScript types. For our database, we chose **Supabase**, a Postgres-based database that provides relational storage for profiles, friendships, tables, hand history, and adverts. For authentication, we will use **Supabase Auth** for account creation, login, and password handling (server verifies user tokens). For hand evaluation, we have chosen to use an open source poker solving library. We've tested its ability to rank hands to determine winners (with ties at showdown), and we verified its ability to handle rank, tie, wheel straight, and kicker cases. For our configuration, we will use **dotenv** to keep API keys and secrets out of the source code. For our version control, we will use **Git** and **GitHub**, as it will allow us to create feature branches and pull request reviews. For our UI sketches and mockups we decided to use **Figma** as it provides a high flexibility. We will use **Jest** to run our tests for both the front and the back end, with the **React Testing Library** for components and **Supertest** for REST routes. We'll test the Socket.IO behavior by connecting test clients to a test server instance, and we'll use Jest's built-in coverage report.

4.4 Coding Standards

Each group member will add comments to clarify their code when appropriate. The `any` type will be avoided in TypeScript. Code files will be named with singular capitalized words, and database tables and columns will use singular snake case names. All group members will meet at least once per sprint. Each sprint, we will use the following merge flow: feature branch → develop branch (requires 1 PR) → main branch (requires 2 PRs). These branch protection rules encourage code reviews and help avoid messy repository states. No restrictions apply while a member is working on their own feature branch. For each merge to develop, 100% of unit tests must pass and there must be at least 80% code coverage. Each test must have a clearly defined purpose, and coverage will be visible through CI/CD workflows set up with GitHub Actions. When a bug is discovered later on, tests should be created for it to prevent regression in the future. Every pull request must be approved by a teammate other than its author, and no one may push directly to the develop or main branches. Branches will be named by the issue number and feature (for example, `feature/13-login-page`), and commit messages will also contain the issue number, be short, and describe the change.

4.5 Data

The database is built around players and the games they play. The user table is the center, and each user has a username, a chip balance, and a few things like a daily reward streak. Players sit at a table, which is a game room with a host, a buy-in, a max number of players, and betting rules. The **table_players** table connects users to tables and tracks which seat they have and how many chips they brought. Each round of play at a table is saved in the match table, which stores the pot and who won. **match_players** lists everyone in that match and how much they won or lost, and **match_log** records every move they

made, like bet, call, raise, fold, check, or draw, so a game can be replayed later. Around this core are the extras: friendship links two users together, **shop_item** and **user_inventory** let players buy and equip things with chips, advert and **ad_claim** let players earn chips by watching ads, and event handles special bonuses. **chat_message** stores table chat, **support_ticket** stores help requests, and **balance_log** keeps a history of every chip change so nothing gets lost.

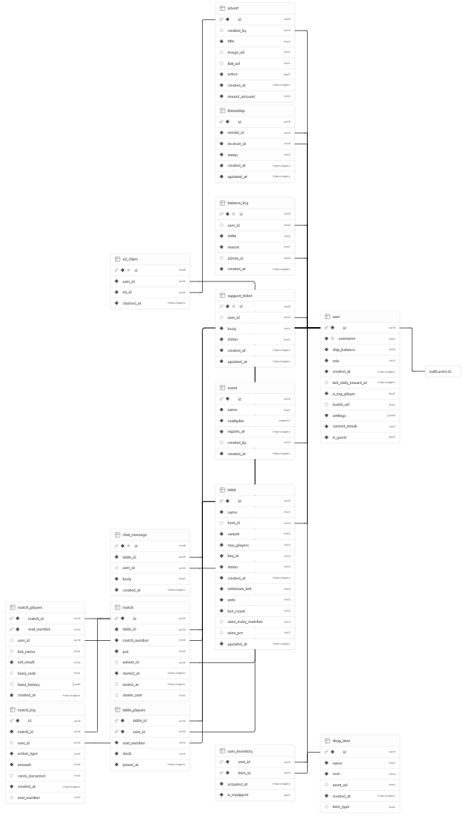
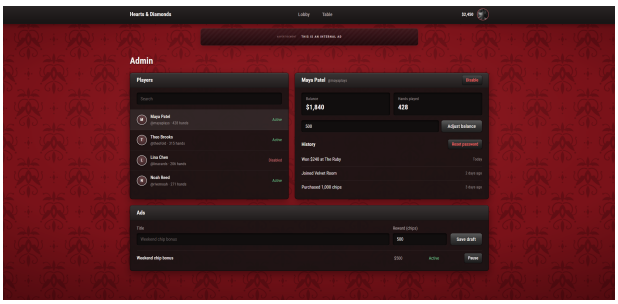


Figure 2: ER diagram of Database Schema

Table	Essential Columns
user	id, username, balance, role, current_streak
friendship	id, sender_id, receiver_id, status
table	id, name, host_id, max_players, buy_in, minimum_bet, status
table_players	table_id, user_id, seat_number, stack
match	id, table_id, match_number, pot, winner_id
match_players	match_id, seat_number, user_id, net_result, hand_rank
match_log	id, match_id, user_id, action_type, amount
shop_item	id, name, cost, item_type
user_inventory	user_id, item_id, is_equipped
advert	id, title, link_url, active, reward_amount
ad_claim	id, user_id, ad_id, claimed_at
event	id, name, multiplier, expires_at
chat_message	id, table_id, user_id, body
support_ticket	id, user_id, body, status
balance_log	id, user_id, delta, reason

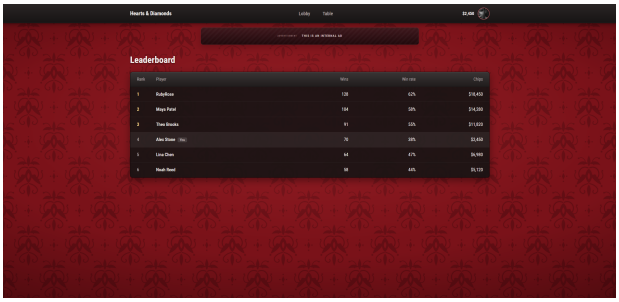
4.6 UI Mocks

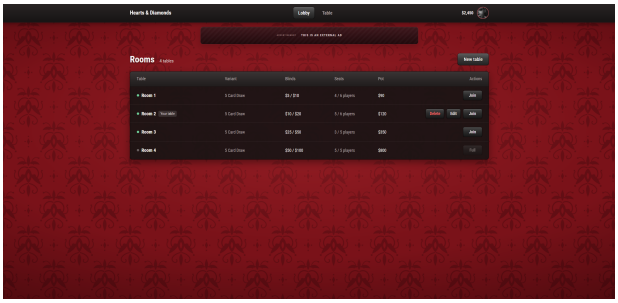
All pages will have a mature red theme. On the gameplay page, a red oval in the middle will act as the poker table. For players, the UI will include a login and sign up page, a page for joining and creating lobbies, the game room, a user profile, and, as a stretch goal, a shop page. For admins, the UI will include a login page, a settings page, a page for spectating rooms, an admin profile, a shop page, and a page for searching through players' profiles.

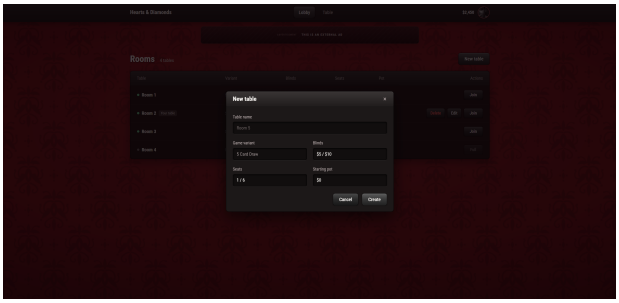


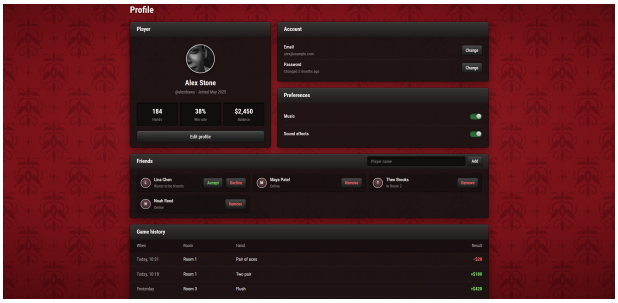




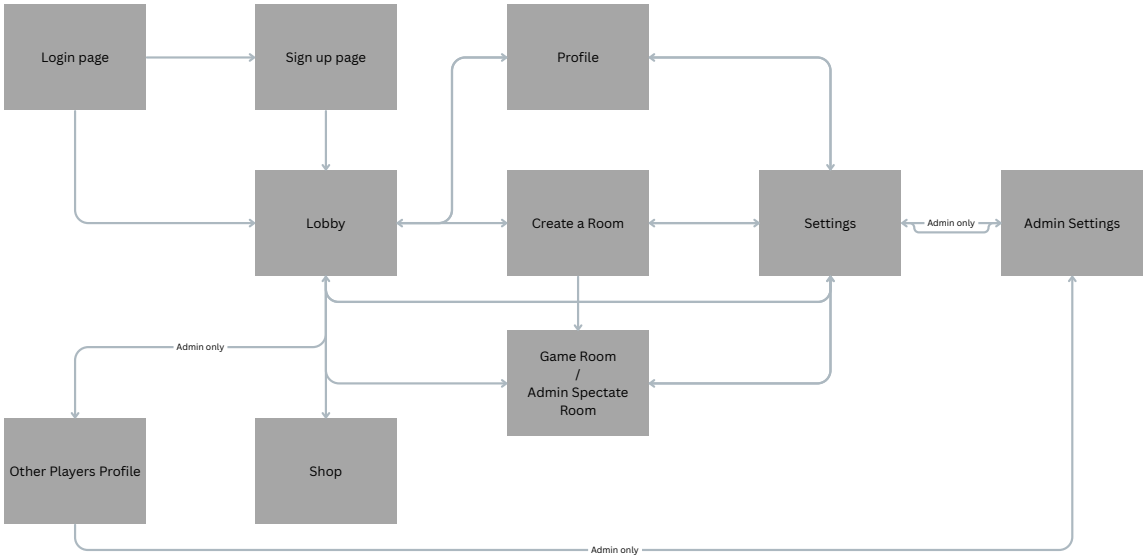












5 Iterations

5.1 Iteration Planning

5.2 Iteration/Sprint 1

5.2.1 Planning

We had a meeting to discuss who would prefer to work on each part of the assignment based on each member's interests and the story priorities. Our goal is to keep within a reasonable scope for our first iteration while also getting the more core features out of the way first.

It.	Dates	Stories	Points Planned	Points Done
1	10/07 - 10/26	S1 Game Initialization, S2 Game Betting, S3 Game Card Draw, S4 Game Ending, S6 Sign Up, S7 Sign In/Sign Out, S9 Guest Player, S12 Room Creation, S14 Room Joining	16	16
2	10/27 - 11/05	S5 Bot Play, S10 View Balance, S11 View My History, S13 Room Settings, S15 Chat Feature, S17 Advertisements (Player), S25 View All Profiles, S29 Advertisements (Admin)	20	20
		Total:	36	36

Table 3: Iteration Planning for Incremental Deliveries

The 3 main features we wanted to focus on were the game engine (dealing, betting, drawing, and determining a winner), rooms (creation and joining), and accounts (sign up, sign in/out, and guest access). Charles took Game Initialization and Game Betting, Sean took Game Card Draw and Game Ending, Heloisa took Sign Up, Sign In/Sign Out, and Guest Player, and Cyrus took on Room Creation and Rooms, giving each member at least 4 story points. Almost every other future story we have planned, such as chat, friends, the leaderboard, and bots, depends on players being able to log in, join a room, and play a hand, so these features are higher priority.

Overall, we plan on completing 16 story points between 4 members and 3 feature areas. We will start by building individual modules for frontend and backend features, following our design plans. We will then integrate our modules as we work through the sprint.

5.2.2 Work Done

Charles completed Game Initialization and Game Betting. Players put in their antes, get dealt 5 cards, and can check, call, bet/raise, or fold during a betting round. Buy-ins are not taken from a real balance yet, since there is no account or currency logic connected.

Sean completed Game Card Draw and Game Ending. Players can pick 0 to 5 cards to discard and get new ones from the deck, and there is a button for it in the UI. At the end of a hand, all hands are compared with a hand evaluator, ties split the pot, and the winner is shown on screen. It also handles players leaving in the middle of a hand. Winnings are not paid out to a real balance yet, since there is no account or currency logic connected.

Cyrus completed Room Creation and Room Joining. Players can create a table with configured settings, see a list of open tables in the lobby, and join or leave a table, with updates sent to everyone at the table. Once enough players join a table, a hand starts on its own. For now, there is dropdown to switch between which fake user account you are acting as.

Heloisa completed Sign Up, Sign In/Sign Out, and Guest Player. Users can create an account, sign in and out, or play as a guest. And new accounts and guests get a row in our user database table. Accounts are not wired to gameplay or rooms just yet.

5.2.3 Testing Coverage

We think our coverage is decent for a first sprint. The backend is at about 93% lines across 150+ tests, and the frontend is at about 84% lines across 46 tests. And all tests pass. The parts that matter most are the parts we tested the most: the game phases (betting, dealing, drawing, turns, and ending a hand), which are near 100%. What was also important were the DAOs, the routers, and the socket code.

Most of the missing coverage comes from code that is not hooked up yet. Sign up and sign in work through our database, but they are not connected to rooms or the game yet, so some of those are at 0%. Those files are going to change considerably once we connect accounts to things next sprint so there is little need to have tests for them at this moment.

We would like to improve `GameService.ts` the most, since the draw phase is tested in `Drawer.ts` but not through the service itself. But overall as long as the game logic and the data layer are solid we are happy. Our plan going forward is to add tests for the auth flow and to keep new features at around the same coverage as now.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	82.05	84.81	81	83.8	
app	72.5	66.66	88.88	80.55	
page.tsx	85.29	66.66	88.88	93.54	41-42
supabaseClient.ts	0	100	100	0	1-7
app/components	100	50	100	100	
AuthBrand.tsx	100	50	100	100	15
app/dashboard	0	100	0	0	
page.tsx	0	100	0	0	1-5
app/signup	0	0	0	0	
page.tsx	0	0	0	0	3-108
component	92.34	89.18	86.2	94.41	
ActionBar.tsx	95.65	100	90	94.44	37
DevUserSwitch.tsx	100	100	100	100	
GameTable.tsx	91.04	84.61	93.33	96.66	51,81
NavBar.tsx	91.66	88.88	100	100	16
NewTableForm.tsx	87.87	100	60	87.5	53,63-71
PlayerHand.tsx	100	50	100	100	12
PlayingCard.tsx	100	40	100	100	16-21
TableList.tsx	92.5	100	83.33	92.3	49-64
service	97.14	96	100	98.27	
CurrentTableService.ts	100	100	100	100	
GameService.ts	100	100	100	100	
SocketService.ts	100	100	100	100	
TableService.ts	100	100	100	100	
UserService.ts	85.71	88.88	100	91.66	19
util	100	100	100	100	
Card.ts	100	100	100	100	

Test Suites: 13 passed, 13 total
Tests: 46 passed, 46 total
Snapshots: 0 total
Time: 4.315 s

(a) Frontend

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	92	85.89	91.8	93.34	
src	34.37	0	16.66	33.33	
app.ts	100	100	100	100	
proxy.ts	0	0	0	0	1-45
src/config	90	83.33	100	100	
Supabase.ts	90	83.33	100	100	11
src/controller	94.95	93.65	85	98.16	
GameController.ts	92.85	96	75	95	13-18
TableController.ts	96.1	92.1	91.66	100	45,51,91
src/dao	98.85	95.23	100	100	
BalanceLogDAO.ts	100	100	100	100	
GameDAO.ts	100	100	100	100	
MatchDAO.ts	100	100	100	100	
TableDAO.ts	100	100	100	100	
TablePlayersDAO.ts	95.83	85.71	100	100	38,50
UserDAO.ts	100	100	100	100	
src/game	98.05	93.06	100	98.99	
Betting.ts	100	98.52	100	100	45
Dealer.ts	88.63	57.14	100	96.96	53
Deck.ts	100	0	100	100	18
Drawer.ts	100	100	100	100	
Ender.ts	98.87	96.8	100	98.74	281-282
Turn.ts	100	100	100	100	
src/route	100	100	100	100	
GameRouter.ts	100	100	100	100	
TableRouter.ts	100	100	100	100	
src/route/auth	0	0	0	0	
route.ts	0	0	0	0	1-17
src/services	87.87	79.16	95.23	92.2	
GameService.ts	87.87	79.16	95.23	92.2	112,137-142
src/socket	100	83.33	100	100	
io.ts	100	83.33	100	100	22

Test Suites: 16 passed, 16 total
Tests: 155 passed, 155 total
Snapshots: 0 total
Time: 3.047 s

(b) Backend

Figure 3: Sprint 1 Test Coverage

5.2.4 Retrospective & Reflection

The main issue in this iteration was that we had no foundation to build on. We had to start working on our own features without any shared context or established way of doing things, so it was really messy figuring things out. This led to a lot of duplicate functionality, and it didn't help that the stories had at least some dependence on each other in order to become ready to showcase. Over time we pulled things from each other's branches and slowly built some common ground for the project structure.

Another issue was getting everything set up under the tight schedules of our group members. All of us were going through midterms, and one of us was also preparing for a theatre production at Loyola for a big chunk of the sprint. However, by communicating well and meeting to work through these problems, we were able to sort out the workload. Setting up the front end was another challenge, and it took a fair amount of struggle, googling, and agent help to understand it. Between the time constraints and complexity of the stories, we didn't perfect every detail of the stories, but we think we got the most important parts done.

Something we all learned this sprint was how hard it is to conceptualize how the frontend and backend work together, especially since most of us had little to no experience building interfaces or calling external APIs. We also learned a lot about TypeScript syntax, like using our own types for things like legal game actions. Learning how to use Socket.IO was a fun part of the sprint, since it was an elegant alternative to the plain WebSockets that some of us are used to. Lastly, we learned how to use MVC properly by keeping each layer separate with its own responsibilities.

5.3 Iteration/Sprint 2

5.3.1 Planning

In this iteration, we focused on building out features for both players and admins so that we are able to increase the functionality of the application. The core features we wanted to work on were advertisements (both admin and player side), bot play, chat, viewing balance and history, and room settings.

Charles took Chat Feature, Sean took Bot Play, Cyrus took Advertisements (Player), Advertisements (Admin), and View All Profiles, and Heloisa took Room Settings, View My History, and View Balance, giving each member at least 4 story points.

Overall, we planned on completing 20 story points between 4 members.

5.3.2 Work Done

Charles completed Chat Feature. Each table now has a live chat, and messages are sent over Socket.IO and saved to the database so that recent messages load when a player opens the table. Charles also improved the table's look, adding card animations and a highlight on whose turn it is.

Sean completed Bot Play. Bots can now be added to the game, and they automatically draw, discard, and bet. The bots use a "happiness" value to determine how much they want to bet. They also have a chance of "bluffing," which makes them more confident in calling and changes their discarding patterns slightly.

Cyrus completed Advertisements (Player), Advertisements (Admin), and View All Profiles. Admins can create ads, and active ads show up in a banner. Players can watch an ad and have its reward added to their balance. Admins can also search to see each player's balance, hands played, net result, and game history.

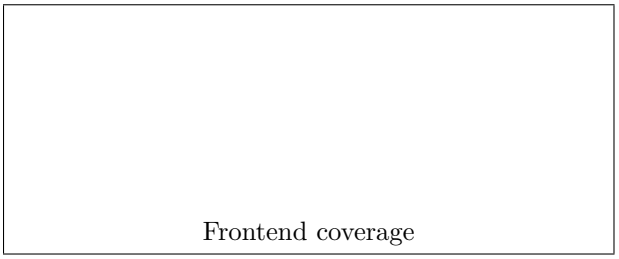
Heloisa completed Room Settings, View My History, and View Balance. Players can now see their balance in the nav bar. They can also change a room's settings, like the room name, the number of bots, and turning chat and the turn timer on or off. Players can also look back at their past games to see how they have done over time.

5.3.3 Testing Coverage

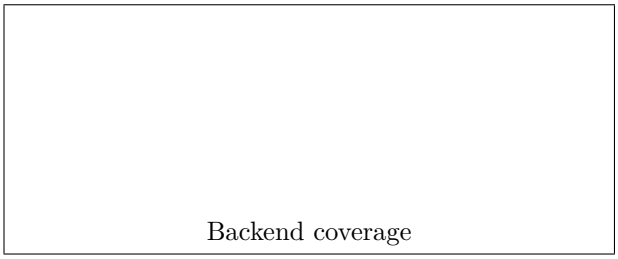
The backend is at about **XX**% lines across **XX** tests, and the frontend is at about **XX**% lines across **XX** tests. Every new feature this sprint came with its own tests, like the ad, admin, chat, and bot tests.

The biggest change in testing from last iteration is that we moved our tests out of the separate test folders and next to the files they test. This makes it easier to see which files have tests and which ones don't.

Our plan going forward is the same as last sprint, which is to keep new features at around the same coverage and to add more tests for the auth flow.



(a) Frontend



(b) Backend

Figure 4: Sprint 2 Test Coverage

5.3.4 Retrospective & Reflection

This sprint was challenging for our group due to the proximity to exams and Halloween, as well as each of us handling our own uniquely busy schedules, but we still managed to communicate well and make a lot of progress on our app. One issue that occurre was overlapping files in the work done by Charles and Cyrus with some of the UI and overall design. However, we fixed this through communication and merging.

During this iteration, working with chat was fun because it allowed us to learn more about and test real-time features. Along with this, it was also interesting making decisions for chat features and storage, as well as setting up rate limits, as we could see how this process could grow much more complex on a larger scale.

It was also really cool to see our game take shape during this iteration. Before, the game felt somewhat simple, but now it feels like we all have our own personal touches on the project, which has been a rewarding part of the process. One of the coolest features that we're excited to have is Bot Players, as this will allow us to play the game more ourselves in our free time and also have it be user tested easily.

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]